

实验五：技能系统集成

实验五：技能系统集成

实验目的

1. 理解技能系统的概念和设计理念
2. 掌握技能加载和匹配机制
3. 学习 NOTICE 技能的实现方法
4. 实现技能与 LLM 的协同工作

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

chat_client.py - 技能系统客户端 - list_available_skills(): 加载可用技能列表 -
load_skill_content(): 加载技能详细内容 - anythingllm_query(): 查询文档仓库 -
stream_llm_response(): 流式响应 - main(): 主函数，集成技能调用

test_skill.py - 技能测试脚本 - 测试技能加载功能 - 验证技能内容读取

2. 技能加载机制 技能目录结构

```
.agents/  
├── skills/  
│   └── notice/  
│       └── SKILL.md
```

技能文件格式 (YAML front matter)

```
---  
name: notice  
description: 用于撰写、修改、润色通知  
---
```

通知撰写技能说明...

技能加载实现

```
def list_available_skills():
    skills = []
    SKILLS_DIR = os.path.join(os.path.dirname(BASE_WORK_DIR), '.agents', 'skills')

    for skill_dir in os.listdir(SKILLS_DIR):
        skill_path = os.path.join(SKILLS_DIR, skill_dir)
        if os.path.isdir(skill_path):
            skill_file = os.path.join(skill_path, 'SKILL.md')
            if os.path.exists(skill_file):
                with open(skill_file, 'r', encoding='utf-8') as f:
                    content = f.read()

                if content.startswith('---'):
                    end_index = content.find('---\n', 3)
                    if end_index != -1:
                        front_matter = content[3:end_index].strip()
                        # 解析 YAML 获取 name 和 description
```

3. NOTICE 技能实现 技能匹配

```
skill_to_call = None
for skill in skills:
    skill_name = skill['name']
    if skill_name == "notice":
        notice_keywords = ["通知", "放假", "公告", "撰写通知", "修改通知", "润色通知"]
        if any(keyword in text_lower for keyword in notice_keywords):
            skill_to_call = skill_name
            break
```

通知生成流程

1. 提取部门（使用正则表达式）

```
department = "XX 部"
dept_match = re.search(r'(?<撰写)(?<修改)(?<润色)(\S+部)', user_input)
if dept_match:
    department = dept_match.group(1)
```

2. 强制固定开头

```
fixed_start = f"{department}通知\n\n"
```

```
# 3. 给模型的指令
notice_instruction = f"""【强制规则，必须 100% 遵守】
你现在需要撰写一份正式通知，** 开头和结尾已经固定，你只需要写中间的正文内容 **：

固定开头：{fixed_start}
固定结尾：特此通知。

你的任务：
1. 只写通知的正文内容
2. 正文内容要正式、规范
3. 不要写任何额外的解释
"""

# 4. 强制拼接开头 + 正文 + 结尾
final_response = f"{fixed_start} {cleaned_body} \n\n特此通知。"
```

实验结果

功能验证

功能	chat_client.py	test_skill.py
技能列表加载	正确	正确
技能内容读取	正确	正确
NOTICE 技能调用	正确	错误
文档仓库查询	正确	错误
流式响应	正确	错误

输出示例

```
=====
AI 智能体（支持技能系统）
=====

提示：
- 输入消息与AI对话
- 系统会自动识别可用技能并调用
- 聊天超过5轮或上下文超过3k时会自动压缩
- 输入 'quit' 或 'exit' 退出

=====

连接到：http://127.0.0.1:1234/v1
使用模型：qwen/qwen3.5-2b
=====

[系统] 已加载 1 个技能
```

[系统] 技能列表: [{"skills": [{"name": "notice", "description": "用于撰写、修改、润色通知"}]]

你：人事部通知大家明天放假

【系统】检测到技能调用：notice

【系统】已设置通知开头：人事部通知

AI：人事部通知

因公司年度团建活动安排，现通知如下：

一、放假时间：明日（X月X日）全天放假一天

二、注意事项：

1. 请各部门做好放假前的安全检查工作
2. 值班人员请按时到岗
3. 放假期间如有紧急事务，请联系各部门负责人

特此通知。

[耗时：2.35秒]

实验总结

实验成果

1. 实现了技能系统框架，支持动态加载技能
2. 实现了 NOTICE 技能，支持通知撰写
3. 实现了技能匹配和调用机制
4. 实现了输出格式控制，确保通知格式正确

技术要点

1. 技能发现：通过文件系统扫描自动发现可用技能
2. YAML 解析：解析技能文件的 front matter 获取元数据
3. 关键词匹配：通过关键词触发相应技能
4. 格式控制：强制控制输出格式，确保符合业务规范

改进建议

1. 可添加技能优先级机制
2. 可实现技能组合调用
3. 可添加技能参数配置界面
4. 可实现技能市场，支持动态安装技能